DVS Scheduling in a Line or a Star Network of Processors*

Zongxu Mu and Minming Li

Department of Computer Science, City University of Hong Kong
mike.mu@student.cityu.edu.hk, minmli@cs.cityu.edu.hk

Abstract. Dynamic Voltage Scaling (DVS) is a technique which allows the processors to change speed when executing jobs. Most of the previous works either study single processor or multiple parallel processors. In this paper, we consider a network of DVS enabled processors. Every job needs to go along a certain path in the network and has a certain workload finished on any processor it goes through before it moves on to the next processor. Our objective is to minimize the total energy consumption while finishing every job before its deadline. Due to the intrinsic complexity of this problem, we only focus on line networks with two nodes and a simple one-level tree network (a star). We show that in some of these simple cases, the optimal schedule can be computed efficiently and interleaving is not needed to achieve optimality. However, in both types of networks, how to find the optimal sequence of execution remains a big challenge for jobs with general workloads.

1 Introduction

Energy efficiency is always a primary concern for chip designers not only for the sake of prolonging the lifetime of batteries which are the major power supply of portable electronic devices but also for the environmental protection purpose when large facilities like data centers are involved. Currently, processors capable of operating at a range of frequencies are already available, such as Intel's SpeedStep technology and AMD's PowerNow technology. The capability of the processor to change voltages is often referred to in the literature as DVS (Dynamic Voltage Scaling) techniques. For DVS processors, since energy consumption is at least a quadratic function of the supply voltage (which is proportional to CPU speed), it saves energy to let the processor run at the lowest possible speed while still satisfying all the timing constraints, rather than running at full speed and then switching to idle.

One of the earliest theoretical models for DVS was introduced by Yao, Demers and Shenker [20] in 1995. They assumed that the processor can run at any speed and each job has an arrival time and a deadline. They gave a characterization of the minimum-energy schedule (MES) and an $O(n^3)$ algorithm for computing

* This work was fully supported by a grant from the Research Grants Council of the Hong Kong Special Administrative Region, China [Project No. CityU 124411].

D.-Z. Du and G. Zhang (Eds.): COCOON 2013, LNCS 7936, pp. 101–113, 2013.
© Springer-Verlag Berlin Heidelberg 2013

it, which was improved to $O(n^2 \log n)$ in [16]. No special assumption was made on the power consumption function except convexity. Several online heuristics were also considered including the Average Rate Heuristic (AVR) and Optimal Available Heuristic (OPA). Under the common assumption of power function $P(s) = s^\alpha$, they showed that AVR has a competitive ratio of $2^{\alpha-1}\alpha^\alpha$ for all job sets. Thus its energy consumption is at most a constant times the minimum required. Later on, under various related models and assumptions, more algorithms for energy-efficient scheduling have been proposed.

Bansal et al. [4] further investigated the online heuristics for the model proposed by [20] and proved that the heuristic OPA has a tight competitive ratio of α^α for all job sets. For the temperature model where the temperature of the processor is not allowed to exceed a certain thermal threshold, they showed how to solve it within any error bound in polynomial time. Chan et al. [5] investigated a slightly more realistic model where the maximum speed is bounded. They proposed an online algorithm which is $O(1)$-competitive in both energy consumption and throughput. More works on the speed bounded model can be found in [3][6][14]. Ishihara and Yasuura [11] initiated the research on discrete DVS problem where a CPU can only run at a set of given speeds. They solved the case when the processor is only allowed to run at two different speeds. Kwon and Kim [13] extended it to the general discrete DVS model where the processor is allowed to run at speeds chosen from a finite speed set. They gave an algorithm for this problem based on the MES algorithm in [20]. Later on, Li et al. [15] improved the time complexity to $O(dn \log n)$ where d is the number of available speeds. When the CPU can only change speed gradually instead of instantly, [9] discussed about some special cases that can be solved optimally in polynomial time. Recently, Wu el al. [19] showed that the optimal schedule for jobs with agreeable deadlines can be computed in polynomial time. Pruhs et al. [18] introduced profit into DVS scheduling. They assume that the profit obtained from a job is a function on its finishing time and on the other hand money needs to be paid to buy energy to execute jobs. They give a lower bound on how good an online algorithm can be and also give a constant competitive ratio online algorithm in the resource augmentation setting. A survey on algorithmic problems in power management for DVS can be found in [1][10].

Most recently, the idea of combining DVS processors and networks was explored by Andrews et al. [2]. They studied the problem of energy efficient routing in the network. They assumed that each link e in the network has an associated energy cost function $f_e(s)$ where s is the demand routed through e. The goal is to minimize the total energy consumption on links while delivering all the demands from their respective sources to the destinations. They designed an approximation algorithm with constant ratio when $f_e(s) = \mu s^\alpha$ where μ and α are fixed constants.

Motivated by the above model, in this paper, we try to explore another combination of DVS processors and networks where each node is a DVS processor and the network is able to handle multiple jobs following different paths in the network.

The remaining paper is organized as follows. In Section 2, we introduce the problem formulation. Section 3 studies line networks and Section 4 studies star networks. Finally, we conclude our paper in Section 5. Due to space limit, some proofs are omitted.

2 Problem Formulation

We use a directed graph $G = (V, E)$ to represent the network of processors. There is a special node s having directed edges to some or all the other nodes and also a special node t having directed edges from some or all the other nodes. Every node $v \in V$ other than s and t represents one processor. Edge $e = (u, v)$ means that jobs can be passed from processor u to processor v. There is a set of jobs $J = \{J_1, J_2, \ldots, J_n\}$ to be processed in this network of processors. Every job J_i is characterized by an arrival time r_i, a deadline (or delay bound) d_i, a path P_i on G, $s \to v_1^i \to v_2^i \to \ldots \to v_{k_i}^i \to t$ and a set of workloads $\{w_{i,v_1^i}, \ldots, w_{i,v_{k_i}^i}\}$ which means that J_i needs to go through every processor on the path in sequence and needs w_{i,v_m^i} CPU cycles on processor v_m^i; when J_i reaches t, the time should be less than d_i. To make the formulation more concise, we normalize d_i so that $\max d_i = 1$.

Each processor has the ability to adjust its speed when executing jobs. If the processor continuously executes a job with the requirement of C CPU cycles at speed s, then the processor needs C/s time to finish its responsibility for the job. The most important element in DVS scheduling is the speed-to-power function $P(s)$ which represents the amount of energy consumed per unit time if the processor runs at speed s. In real systems, $P(s)$ is convex and usually has the form $P(s) = s^\alpha$ where α is 2 or 3. We want to produce a schedule for all the processors so that every job is following the correct sequence of processors and is finished by its deadline. Among all the feasible schedules, we want to find a schedule with minimum energy consumption. To formalize the problem description, we give the following definition:

A schedule S for job set J consists of a schedule for every processor v consisting of two functions $(s_v(t), job_v(t))$ which define, respectively, the processor speed and the job being executed at time t on processor v.

A schedule is feasible if for every job J_k, the following conditions are satisfied:

(1) $s_v(t) \geqslant 0$;

(2) $job_v(t) \in \{1, 2, ..., n\}$ where n is the number of jobs in the job set J;

(3) $\int_{r_i}^{d_i} s_v(t)\delta(k, job_v(t))dt = w_{k,v}$, where $\delta(i, j) = 1$ if $i = j$ and $\delta(i, j) = 0$ otherwise;

(4) $\max\{t | job_u(t) = k\} \leq \min\{t | job_v(t) = k\}$ if u is before v in the path P_k.

The energy consumption of a schedule $S = \{s_v(t), job_v(t)\}$ is defined as $E(S) = \sum_{v \in V} \int_0^1 P(s_v(t))dt$ with the power function given as $P(s) = s^\alpha$. Hence, the optimization problem we study is:

$$\begin{aligned}
&\min \textstyle\sum_{v\in V}\int_0^1 P(s_v(t))dt\\
&s.t.\ \ s_v(t)\geqslant 0;\\
&\qquad job_v(t)\in\{1,2,...,n\}\\
&\qquad \textstyle\int_{r_i}^{d_i} s_v(t)\delta(k,job_v(t))dt = w_{kv};\\
&\qquad \max\{t|job_u(t)=k\}\leq \min\{t|job_v(t)=k\}\text{if } u \text{ is before } v \text{ in the path } P_k
\end{aligned}$$

Since now the arrival time of a job at a certain processor is affected by how fast it is done on previous processors, the energy minimization problem is not simply the combination of single processor scheduling problems. New insights need to be identified in order to solve this problem. In the following discussion, we assume $\alpha = 2$.

3 Line Networks

We start by investigating a special case when the network of processors forms a line (called line network). In other words, the graph is $G = (V, E)$ where $V = \{s, v_1, v_2, \ldots, v_m, t\}$ and $E = \{(s, v_1), (v_m, t), (v_i, v_{i+1})|1 \leq i \leq m-1\}$ and $m > 1$. Every job needs to go through the only path in graph G.

In the problem, we require all jobs to be processed and pass through the network within the period $[0, d]$. More precisely, we have $r_k = 0$ and $d_k = d$ for all $J_k \in J$. To simplify the notation, we use j to represent v_j^i, which is the jth node in the line network. Thus, $w_{i,j}$ represents the workload of job i on processor j. Under the above setting, it is easy to see that the optimal schedule will use only one speed when executing $w_{i,j}$. Therefore, we use $s_{i,j}$ to represent the processing speed of $w_{i,j}$, $t_{i,j}$ to represent the time spent on processing $w_{i,j}$ and $E_{i,j}$ to represent the energy consumption of executing $w_{i,j}$. Thus, the total energy consumption is $\sum_{i,j} E_{i,j}$ (we will use E to represent it afterward), which is our target to minimize.

The line network scheduling problem in our model resembles flowshop scheduling model with an extra twist of speed change. A flow shop is an environment where multiple operations required for a job have to be done in the same order [17]. In other words, the jobs have to follow the same route of machines. In the makespan flowshop scheduling model, each job needs to pass L stages in order, with each stage consisting of $M^{(l)}$ processors, and the objective is to minimize the makespan of all jobs. As presented by Johnson in [12] which is widely known as Johnson's Rule, the flowshop scheduling problem can be solved to optimality when $L = 2$. However, with $L > 2$, such optimality is no longer achievable. Indeed, Garey et al. have proved that optimal scheduling is already NP-hard with $L = 3$ [7], i.e. , the $F3||C_{max}$ problem, using the standard notation proposed by Graham et al. [8].

However, our model is indeed different from makespan flowshop scheduling because the processing time of a job on one processor is not fixed due to processor's ability to change speed and our objective is minimizing the energy consumption instead of minimizing makespan. For the makespan flowshop model, a processor

may be idle in the middle of an optimal schedule; however, it is not the case for the model we study because the energy consumption can always be reduced by extending the processing time of one or both of the jobs before and after the idle period. Therefore, it may be necessary to operate at different speeds in order to achieve energy minimization. This imposes further complexity to the problem.

In our model, a schedule is called an optimal schedule (or min-energy schedule) if its energy consumption is the minimum among all the feasible schedules. We are interested in efficient scheduling algorithms where given a job set J and a network of processors represented by G, we aim to find a schedule S with the minimum energy consumption while achieving the delay bound of every job.

We first define non-interleaving schedules and interleaving schedules.

Definition 1. *A non-interleaving schedule is a schedule satisfying:*

1. *For all $i'(1 \leqslant i' \leqslant n)$, if $w_{i',1}$ is the jth finished workload among all $w_{i,1}$, then $w_{i',2}$ is also the jth finished workload among all $w_{i,2}$. It guarantees that all jobs will be processed in the same order on both processors.*
2. *Once $w_{i,j}$ starts being processed, the jth processor will not process any other workload until $w_{i,j}$ is completed.*

A schedule violating any of the two conditions is called an interleaving schedule.

The second property implies non-preemption in flowshop scheduling. Together with the first property it leads to permutation in the flowshop.

3.1 Line Network of 2 Processors with 2 Jobs

In this subsection, we will find the optimal schedule for 2 jobs in a line network with 2 processors. Firstly, we argue that we can always find an optimal schedule among non-interleaving schedules. The proof is given in Section 3.2 where we discuss the more general case of energy minimization in a line network of 2 processors with n jobs. Here, we still deal with 2-job case separately for the purpose of elaborating the proving techniques we use for other cases.

If the schedule begins with job J_1, the workload $w_{1,1}$ will be the first to be processed. With no interleaving, the workloads $w_{i,j}$ $(i = 1, 2;\ j = 1, 2)$ will be processed as illustrated by Table 1.

Table 1. Work flow in the case of 2 jobs and 2 processors

Node 1	Node 2
$w_{1,1}$	
$w_{2,1}$	$w_{1,2}$
	$w_{2,2}$

Note that $w_{1,1}$ must finish before $w_{1,2}$ and $w_{2,1}$, while $w_{2,2}$ can only be processed after $w_{2,1}$ and $w_{1,2}$ are finished. In addition, $w_{2,1}$ and $w_{1,2}$ will start and

stop processing at the same time. Otherwise, it is always possible to reduce the energy consumption by extending the one with the later start time or earlier end time, because this will not affect the execution of $w_{1,1}$ and $w_{2,2}$, but will increase the processing time, hence reduce the processing speed of $w_{2,1}$ or $w_{1,2}$.

Note that the speed setting problem is in essence equivalent to a time division problem, where each job is executed with the minimum possible constant speed within its assigned interval. With a workload w executed in a time interval with length t at a fixed speed, the total energy consumption is $s^2 t = \frac{w^2}{t^2} t = \frac{w^2}{t}$.

Therefore, we assume $w_{1,1}$ takes a period of t_1, $w_{1,2}$ and $w_{2,1}$ takes a period of t_2, and $w_{2,2}$ takes a period of t_3, where $t_1 + t_2 + t_3 = d$. Then, the total energy consumption is

$$E = \frac{w_{1,1}^2}{t_1} + \frac{w_{1,2}^2 + w_{2,1}^2}{t_2} + \frac{w_{2,2}^2}{t_3}.$$

We remark that $w_{1,1}$ and $w_{2,2}$ should be done at the same constant speed within the period $d - t_2$ in the optimal schedule. Otherwise, we can reduce the power consumption by speeding up the originally slower one and slowing down the other. Referring to the deduction above, the total energy consumption is

$$E = \frac{(w_{1,1} + w_{2,2})^2}{d - t_2} + \frac{w_{1,2}^2 + w_{2,1}^2}{t_2}.$$

Therefore, to minimize E subject to $0 < t_2 < d$, we need to have

$$\frac{\mathrm{d}E}{\mathrm{d}t_2} = \frac{(w_{1,1} + w_{2,2})^2}{(d - t_2)^2} - \frac{w_{1,2}^2 + w_{2,1}^2}{t_2^2} = 0.$$

Therefore, we get

$$t_2 = \frac{d\sqrt{w_{1,2}^2 + w_{2,1}^2}}{w_{1,1} + \sqrt{w_{1,2}^2 + w_{2,1}^2} + w_{2,2}}.$$

Because

$$\frac{\mathrm{d}^2 E}{\mathrm{d}t_2^2} = \frac{2\,(w_{1,1} + w_{2,2})^2}{(d - t_2)^3} + \frac{2\left(w_{1,2}^2 + w_{2,1}^2\right)}{t_2^3} > 0,$$

for the schedule proposed in Table 1, we can obtain the minimized value of E:

$$E_{\min} = \frac{\left(w_{1,1} + \sqrt{w_{1,2}^2 + w_{2,1}^2} + w_{2,2}\right)^2}{d}.$$

Interestingly, the calculated E_{min} above is exactly the minimum energy consumption in the situation where

- There are 3 processors in the line network, and
- There is 1 job with workloads $w_{1,1}$, w_0 and $w_{2,2}$ respectively on three processors where $w_0 = \sqrt{w_{1,2}^2 + w_{2,1}^2}$.

Similarly, if we begin with job 2, the minimum energy consumption will be

$$\frac{\left(w_{2,1} + \sqrt{w_{2,2}^2 + w_{1,1}^2} + w_{1,2}\right)^2}{d}.$$

Hence, whether having J_1 or J_2 done first will lead to the energy minimization is determined by the relative value of $w_{2,1} + \sqrt{w_{2,2}^2 + w_{1,1}^2} + w_{1,2}$ and $w_{1,1} + \sqrt{w_{1,2}^2 + w_{2,1}^2} + w_{2,2}$.

3.2 Line Network of 2 Processors with n Jobs

Lemma 1. *In a line network of 2 processors with n jobs, an optimal schedule can always be found among non-interleaving schedules.*

Proof. We prove the lemma by showing that it is always possible to turn an interleaving schedule into a non-interleaving schedule without increasing the energy consumption. Therefore, if an interleaving schedule can achieve energy minimization, the minimized energy can be achieved by a corresponding non-interleaving schedule as well.

We number the jobs in a way that $w_{i,1}$ finishes earlier than $w_{j,1}$ for all $i < j$.

Firstly, we consider the schedule on the second processor. For any pair of $w'_{j,2}$ and $w'_{i,2}$ which denote adjacent segments where $i < j$ and $w'_{j,2}$ is processed before $w'_{i,2}$, it is always possible to change their order into $w'_{i,2}$ and then $w'_{j,2}$ without affecting the processing of other workloads. Since $w_{i,1}$ finishes earlier than $w_{j,1}$ with $i < j$, the swapping of the segments will not be constrained by their corresponding segments at the first processor. Moreover, the swapping of segments will not affect the total energy consumed, since they can still be processed at the same speed as before. Therefore, given a finishing order of $w_{i,1}$ for all i at the first processor, the non-interleaving schedule of $w_{i,2}$ will give a no worse result than any interleaving schedule.

Secondly, given the finishing order of $w_{i,1}$ for all i at the first processor, for every $w_{i,1}$, the first processor has to complete $\sum_{j=0}^{i} w_{j,1}$ in order to finish $w_{i,1}$ because the finishing order is pre-defined. The non-interleaving schedule will make sure that every workload is finished as early as possible.

Therefore, changing an interleaving schedule to a non-interleaving one by the way shown above will result in no more energy consumption without delaying the finish time of any job. □

The energy minimization problem of 3 or more jobs is quite similar to that of 2 jobs as discussed in Section 3.1, except that we need to consider more precedence constraints, that is, we can only start processing $w_{i,2}$ if processor 1 finishes processing $w_{i,1}$. In the following, we assume the execution order of the jobs are from J_1 to J_n.

In the optimal schedule, if the finishing time of $w_{i,1}$ is equal to the starting time of $w_{i,2}$, we say this time is a *critical time*. The time has "critical" properties

because it divides the problem into sub-problems, where jobs J_1 to J_i can be analyzed separately from jobs J_{i+1} to J_n. It is easy to argue that workloads in the same segment must be processed using the same speed on the same processor because otherwise we can always slightly shift the workload to make the speed curve flatter without violating the precedence constraint between $w_{i,2}$ and $w_{i,1}$ (In the interior of a segment, the finishing time of $w_{i,1}$ is strictly before the starting time of $w_{i,2}$).

We further define job i to be a critical job if the finishing time of $w_{i,1}$ is a critical time in the optimal schedule. Suppose that $c_1, c_2, \ldots, c_k$ are critical jobs (job J_n must be a critical job because we can always delay the finishing time of workload $w_{n,1}$ to be equal to the starting time of $w_{n,2}$ without increasing the energy consumption. Similarly job J_1 is also a critical job). Then we can use the similar observation as the 2-job-2-processor case to calculate the corresponding minimum energy consumption to be

$$E_{min} = \frac{\left[w_{1,1} + \sum_{i=1}^{k-1} \sqrt{\left(\sum_{j=c_i+1}^{c_{i+1}} w_{j,1} \right)^2 + \left(\sum_{j=c_i}^{c_{i+1}-1} w_{j,2} \right)^2} + w_{n,2} \right]^2}{d} \tag{1}$$

Hence, finding the optimal schedule for a given order of execution becomes finding critical jobs while guaranteeing workloads in the same segment can be executed continuously on two processors (with stable speed on each processor) without violating the constraint between $w_{i,1}$ and $w_{i,2}$. This property is tested in the inequality inside the for loop in Algorithm 1, a dynamic programming algorithm we design to calculate the optimal schedule by guessing critical jobs. The intuition behind this testing is that as long as the proportion of the workloads of the type $w_{i,1}$ till any possible critical time in the middle is less than the proportion of the workloads of type $w_{i,2}$ in the same period, then using a stable speed on the same machine will always finish $w_{i,1}$ earlier than $w_{i-1,2}$, which removes the possibility of critical time in between.

Theorem 1. *Procedure FindOptWork*$(1, n)$ *can compute the optimal schedule when the processing sequence is fixed in* $O(n^3)$ *time.*

Proof. The correctness is already proved in the above discussion. About the time complexity, there are n^2 procedures to be called and each procedure takes $O(n)$ time to finish if we preprocess all the testing in the IF statement in $O(n^3)$ time. Therefore, the overall complexity is $O(n^3)$. □

However, how to find the optimal sequence remains an open question.

3.3 Optimal Sequencing for Jobs with the Same Workload on the Second Processor

In this subsection, we derive the optimal sequence for a line network of 2 processors with n jobs, where jobs have the same workload on the second processor. We

Procedure $FindOptWork(start, end)$
Input: $start$ and end
Output: Optimal equivalent workload during this period

```
Min = ∞
if start == end then
    return 0
end if
for i = start + 1 to end do
    /* testing whether start to i can be a segment in the optimal schedule
    if (Σ_{j=start+1}^{k} w_{j,1}) / (Σ_{j=start+1}^{i} w_{j,1}) ≥ (Σ_{j=start}^{k-1} w_{j,2}) / (Σ_{j=start}^{i-1} w_{j,2}) for all start + 1 ≤ k ≤ i then
        Min = min{Min, sqrt((Σ_{j=start}^{k-1} w_{j,1})^2 + (Σ_{j=start}^{i-1} w_{j,2})^2) +
        FindOptWork(i, end)}
    end if
end for
return Min
```

argue that the ascending order of workloads on the first processor can achieve optimality. We assume that the optimal sequence is $J_1, J_2, \ldots, J_n$.

Lemma 2. *To achieve optimality, $w_{1,1}$ should be the smallest among all $w_{i,1}$.*

Lemma 3. *To achieve optimality, $w_{i,1} \leq w_{i+1,1}$ for $i \in [2, n-1]$.*

Theorem 2. *The ascending order according to $w_{i,1}$ is the optimal sequence.*

Proof. By Lemma 2 and Lemma 3, the ascending order of $w_{i,1}$ is the optimal sequence to process jobs in the two processors. □

4 Star Network

In this section, we extend our analysis to a star network between s the sender and t the receiver as defined below:

Definition 2. *The star network consists of $n+1$ processors, out of which, the sender is connected to one intermediate processor v_1. v_1 is then connected to n nodes $v_2, v_3, \cdots v_{n+1}$, all of which are connected to t. There will be exactly 1 job sent from the sender to the receiver via one of the nodes in $\{v_i | 2 \leqslant i \leqslant n+1\}$ during the time interval $[0, d]$. (We define n as the degree of the star)*

In such a structure, finding the optimal schedule is boiled down to finding an optimal sequence of scheduling on v_1. Hence, our focus in this section is to calculate the optimal sequence given different workload characteristics.

We assume that the jobs are executed in the order of $J_1, J_2, \cdots, J_n$. For a star network, the possible schedule can be seen in Table 2.

Note that $w_{i,2}$ shares the same processing time with $w_{i+1,1}$ and $w_{i+1,2}$ as a whole for $1 \leqslant i < n$. For the case of $n = 2$, the only difference with line networks

Table 2. Processing schedule in a star network

<table>
<tr><td>v_1</td><td>v_2</td><td>v_3</td><td>$\cdots$</td><td>v_{n+1}</td></tr>
<tr><td>$w_{1,1}$</td><td></td><td></td><td></td><td></td></tr>
<tr><td>$w_{2,1}$</td><td rowspan="4">$w_{1,2}$</td><td rowspan="4">$w_{2,2}$</td><td></td><td></td></tr>
<tr><td>$\cdots$</td><td></td><td></td></tr>
<tr><td rowspan="2">$w_{n+1,1}$</td><td rowspan="2">$\vdots$</td><td rowspan="2"></td></tr>
<tr><td></td></tr>
<tr><td></td><td></td><td></td><td></td><td>$w_{n+1,2}$</td></tr>
</table>

is that in the line networks, $w_{1,2}$ shares the processing time with $w_{2,1}$ only and $w_{2,2}$ is processed after both. Therefore, the minimal energy consumption for the star network with $n = 2$ is exactly the same as the following scenario:

- There are 2 processors in the line network,
- There are 2 jobs to be processed: the first one J_1^{line} has $w_{1,1}^{line} = w_{1,1}$ and $w_{1,2}^{line} = w_{1,2}$, and the second one J_2^{line} has $w_{2,1}^{line} = w_{2,1} + w_{2,2}$ and $w_{2,2}^{line} = 0$.

Therefore, the minimal energy consumption

$$E = \frac{\left[w_{1,1} + \sqrt{w_{1,2}^2 + (w_{2,1} + w_{2,2})^2}\right]^2}{d}$$

Note that this energy consumption is further equal to the energy consumption in the situation where

- There are 2 processors in the line network, and
- There is 1 job with workloads $w_{1,1}$ and $\sqrt{w_{1,2}^2 + (w_{2,1} + w_{2,2})^2}$ respectively on the 2 processors

Recursively, for $n \geqslant 2$, the minimal energy consumption can be derived as

$$E = \frac{\left\{w_{1,1} + \sqrt{w_{1,2}^2 + \left[w_{2,1} + \sqrt{w_{2,2}^2 + \left(w_{3,1} + \sqrt{w_{3,2}^2 + \ldots}\right)^2}\right]^2}\right\}^2}{d} \tag{2}$$

where $w_{i,1} + \sqrt{w_{i,2}^2 + \left(w_{i+1,1} + \sqrt{w_{i+1,2}^2 + \cdots}\right)^2}$ is called the equivalent workload from J_i to J_n.

4.1 Optimal Sequencing for Jobs with the Same Workload on the Second Processor

In this subsection, we consider jobs whose workloads on the second processor are all the same. We show the following lemma.

Lemma 4. *In a star network, if all $w_{i,2}$ are the same (we use $w_{x,2}$ to represent it) and $w_{i,1}$ is sorted in increasing order, then in the optimal schedule, $w_{i,1}$ should be done before $w_{j,1}$ if $i < j$ and the minimized energy consumption is*

$$E = \frac{\left\{ w_{1,1} + \sqrt{w_{x,2}^2 + \left[w_{2,1} + \sqrt{w_{x,2}^2 + \left(w_{3,1} + \sqrt{w_{x,2}^2 + \ldots} \right)^2} \right]^2} \right\}^2}{d}.$$

4.2 Optimal Sequencing for Equal-Workload Jobs

In this subsection, we consider jobs whose workloads on the first processor and the second processor are the same, but different jobs can have different workloads. In other words, we have $w_{i,1} = w_{i,2}$ for all $1 \leqslant i \leqslant n$. In the following, we use w_k to represent either $w_{k,1}$ or $w_{k,2}$. Suppose that the optimal sequence is $J_1, J_2, \ldots, J_n$.

Lemma 5. *In an optimal job sequencing for a star network, we have $w_1 \geq w_2$.*

Lemma 6. *In an optimal job sequencing, for any two neighboring workloads w_i and w_{i+1}, we have $w_i \geq w_{i+1}$.*

Theorem 3. *The optimal job sequencing should be in monotonically decreasing order with respect to the workloads.*

Proof. This theorem directly follows by Lemma 5 and 6. □

5 Conclusion

In this paper, we study the energy minimization problem in a network of processors, specifically line networks and star networks. We show that non-interleaving can achieve optimality when the line network contains two processors and the energy optimization problem in fact becomes an interesting sequencing problem which can be solved optimally in some special cases. On the other hand, for star networks, we characterized the optimal schedule when the workloads of jobs on the second level are the same. We also derived the optimal sequence for jobs whose workloads on both levels are the same. Although we assume $P(s) = s^\alpha$ with $\alpha = 2$ for the proof of the above properties, most of the properties can be easily extended for any $\alpha > 1$. For example, Minkowski inequality needs to be used instead of Cauchy inequality in the proof of Lemma 3. How to deal with the other cases remains an open problem. In both types of networks, finding the optimal sequence of execution given arbitrary workloads is a big challenge.

References

1. Albers, S.: Algorithms for Dynamic Speed Scaling. In: STACS 2011, pp. 1–11 (2011)
2. Andrews, M., Fernandez, A., Zhang, L., Zhao, W.: Routing for Energy Minimization in the Speed Scaling Model. In: Proceedings of 29th IEEE International Conference on Computer Communications, pp. 2435–2443 (2010)
3. Bansal, N., Chan, H.-L., Lam, T.-W., Lee, L.-K.: Scheduling for speed bounded processors. In: Aceto, L., Damgård, I., Goldberg, L.A., Halldórsson, M.M., Ingólfsdóttir, A., Walukiewicz, I. (eds.) ICALP 2008, Part I. LNCS, vol. 5125, pp. 409–420. Springer, Heidelberg (2008)
4. Bansal, N., Kimbrel, T., Pruhs, K.: Dynamic Speed Scaling to Manage Energy and Temperature. In: Proceedings of the 45th Annual Symposium on Foundations of Computer Science, pp. 520–529 (2004)
5. Chan, H.L., Chan, W.T., Lam, T.W., Lee, L.K., Mak, K.S., Wong, P.W.H.: Energy Efficient Online Deadline Scheduling. In: Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms, pp. 795–804 (2007)
6. Chan, J.W.-T., Lam, T.-W., Mak, K.-S., Wong, P.W.H.: Online deadline scheduling with bounded energy efficiency. In: Cai, J.-Y., Cooper, S.B., Zhu, H. (eds.) TAMC 2007. LNCS, vol. 4484, pp. 416–427. Springer, Heidelberg (2007)
7. Garey, M., Johnson, D.S., Sethi, R.: The complexity of flowshop and jobshop scheduling. Math. of Oper. Res. 1, 117–129 (1976)
8. Graham, R.L., Lawler, E.L., Lenstra, J.K., Rinnooy Kan, A.H.G.: Optimization and approximation in deterministic sequencing and scheduling theory: a survey. Ann. Discrete Math. 5, 287–326 (1979)
9. Hong, I., Qu, G., Potkonjak, M., Srivastavas, M.B.: Synthesis techniques for low-power hard real-time systems on variable voltage processors. In: Proceedings of the IEEE Real-Time Systems Symposium, pp. 178–187 (1998)
10. Irani, S., Pruhs, K.: Algorithmic Problems in Power Management. ACM SIGACT News 36(2), 63–76 (2005)
11. Ishihara, T., Yasuura, H.: Voltage Scheduling Problem for Dynamically Variable Voltage Processors. In: Proceedings of International Symposium on Low Power Electronics and Design, pp. 197–202 (1998)
12. Johnson, S.M.: Optimal Two- and Three-stage Production Schedules with Setup Times Included. Naval Res. Logist. Quart. 1, 61–68 (1954)
13. Kwon, W., Kim, T.: Optimal Voltage Allocation Techniques for Dynamically Variable Voltage Processors. In: Proceedings of the 40th Conference on Design Automation, pp. 125–130 (2003)
14. Lam, T.W., Lee, L.K., To, I.K.K., Wong, P.W.H.: Energy Efficient Deadline Scheduling in Two Processor Systems. In: Proceedings of the 18th International Symposium on Algorithm and Computation, pp. 476–487 (2007)
15. Li, M., Yao, F.F.: An Efficient Algorithm for Computing Optimal Discrete Voltage Schedules. SIAM Journal on Computing 35(3), 658–671 (2005)
16. Li, M., Yao, A.C., Yao, F.F.: Discrete and Continuous Min-Energy Schedules for Variable Voltage Processors. Proceedings of the National Academy of Sciences USA 103, 3983–3987 (2005)
17. Pinedo, M.: Scheduling: Theory, Algorithms, and Systems. In: Flow Shops and Flexible Fow Shops (Deterministic), 2nd edn., ch. 6, p. 129. Prentice-Hall, Englewood Cliffs (2002)
18. Pruhs, K., Stein, C.: How to Schedule When You Have to Buy Your Energy. In: Serna, M., Shaltiel, R., Jansen, K., Rolim, J. (eds.) APPROX and RANDOM 2010. LNCS, vol. 6302, pp. 352–365. Springer, Heidelberg (2010)

19. Wu, W., Li, M., Chen, E.: Min-Energy Scheduling for Aligned Jobs in Accelerate Model. In: Dong, Y., Du, D.-Z., Ibarra, O. (eds.) ISAAC 2009. LNCS, vol. 5878, pp. 462–472. Springer, Heidelberg (2009)
20. Yao, F., Demers, A., Shenker, S.: A scheduling model for reduced CPU energy. In: Proceedings of the 36th Annual IEEE Symposium on Foundations of Computer Science, pp. 374–382 (1995)